Sparse FGLM for the linear stage of `jv_cover`: C_{cov} 1.27–1.31x cheaper, S 1.28x cheaper, same relations

2026-10-19. Run report: `reports/run-20261018T2138Z/report.pdf`. Class: **engineering** (same algorithm, cheaper linear algebra).

Question

After the `f4_fp` speed-up, the linear stage after F4 is 40 % of one Nagao test (4.8e5 of 1.22e6 mults at p = 101). The derived estimate was 210k–260k for it and C_{cov} 1.22–1.29x cheaper. Does an exact sparse path beat that?

What was built (`solve_zero_dim_mode`, `src/cryptanalysis/jv_cover.rs`)

- Only M_{x_6} is built. Its 44 unit columns are shifts (a target index); the 20 others are stored as sparse vectors.
- Minimal polynomial by Wiedemann: $s_i = u^T M^i v$, $i < 2\delta$, Berlekamp–Massey; up to 4 draws. Degree δ means minimal polynomial = characteristic polynomial, so no root is missed and x_6 separates the points. Else: dense fallback (both costs counted, `lin_fallback`).
- Roots by `PolyRing::roots`. For each root r the eigenvector w of M^T satisfies $w[x_6 b] = r w[b]$, so it is fixed by one unknown per shift chain (20); the dense columns give a 20x20 system of corank one; $x_i = \frac{w[x_i]}{w[1]}$, $x_6 = r$. Corank not one, or a missing degree-1 standard monomial: dense fallback.
- Every candidate is still checked against all input polynomials.
- `JV_LIN=dense|l1a|sparse` (default sparse) selects the arm in one binary. L1a = dense Hessenberg charpoly of M_{x_6} + the same chain recovery.
- All arms draw one `u64` from the caller's rng and use a private rng after that, so downstream randomness is identical across arms. This changes the stream relative to older main (baseline numbers move by under 0.1 %).
- Unit test `sparse_linear_stage_agrees_with_dense` (p = 53, 101; 4–6 variables; 48 systems, planted and random): dense, sparse and L1a return identical solution sets; passes in release.

One decomposition test (ccov, 120 tests per prime)

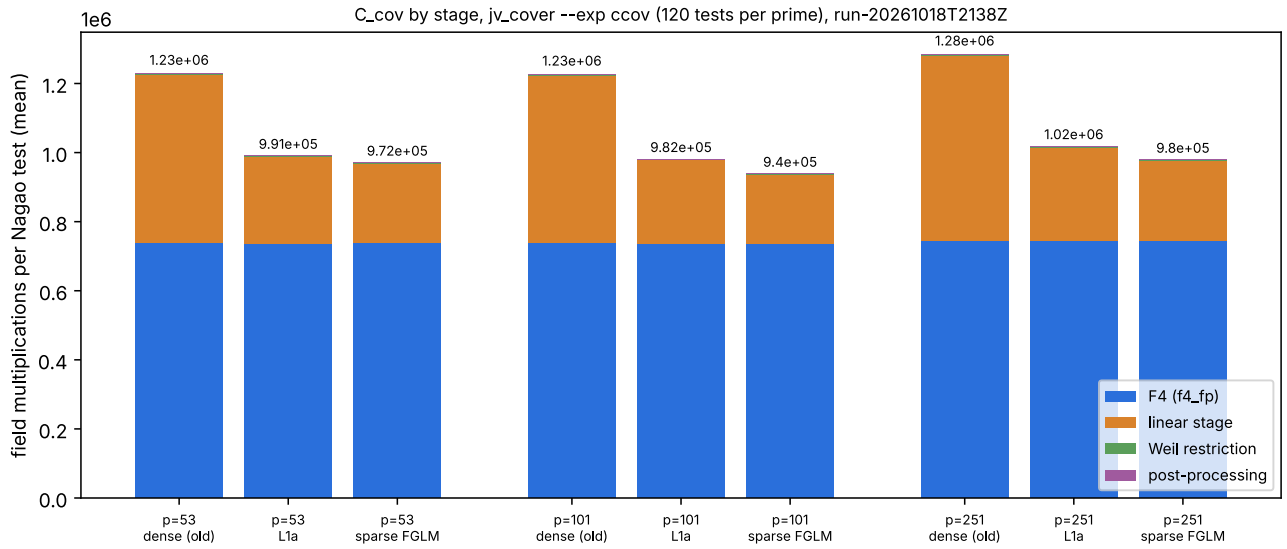


Figure 1: C_{cov} by stage, three arms.

p	planted	lin dense	lin sparse	lin L1a	C_{cov} dense	C_{cov} sparse (ratio)	L1a (ratio)
53	20/20 x3	4.875e5	2.296e5	2.490e5	1.2296e6	9.718e5 (1.265)	9.912e5 (1.241)
101	20/20 x3	4.863e5	1.991e5	2.409e5	1.2274e6	9.403e5 (1.305)	9.820e5 (1.250)
251	20/20 x3	5.347e5	2.304e5	2.690e5	1.2844e6	9.800e5 (1.311)	1.0186e6 (1.261)

Gates, every arm: planted 20/20, mismatches 0, unverified 0, incomplete 0. Dense `nonseparating` 3, 2, 0 (a random ℓ failed); sparse falls back to dense in 4, 0, 1 of 120 tests (p = 53, 101, 251), L1a likewise; the failed attempt is paid in the sparse cost. Linear stage: 2.12x, 2.44x, 2.32x cheaper.

End to end (`dlp`)

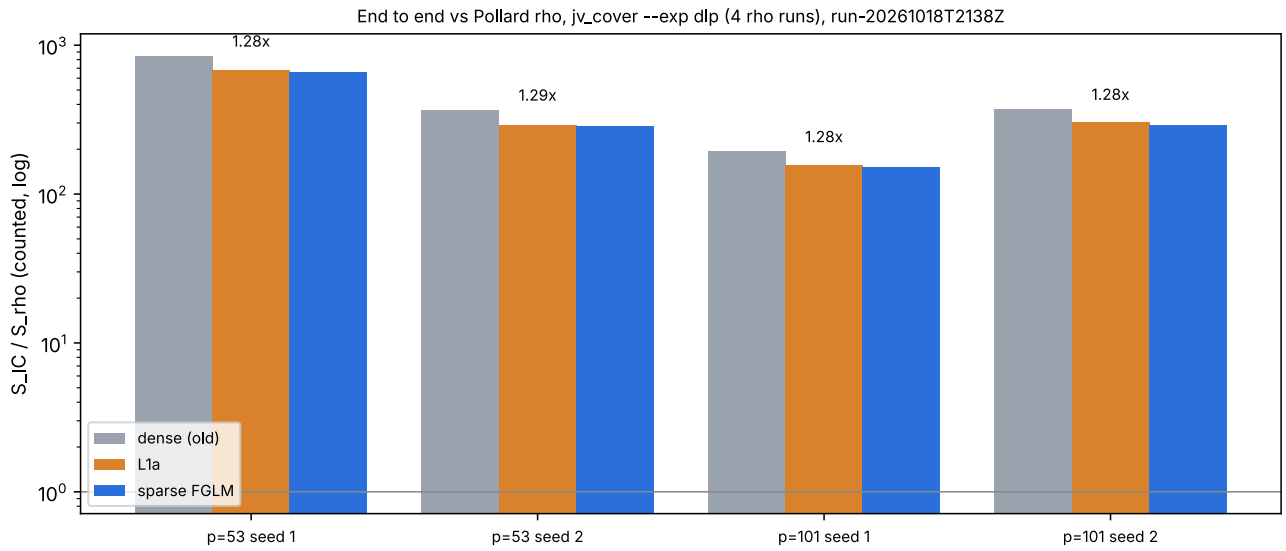


Figure 2: S/S_{rho} by arm.

p, seed	relations	correct	S dense	S sparse	ratio	S/S_{rho} dense	S/S_{rho} sparse (L1a)
53, 1	31	yes	937.2	731.6	1.281	851	664 (683)
53, 2	33	yes	424.5	328.8	1.291	369	286 (293)
101, 1	53	yes	287.8	224.4	1.282	194	151 (157)
101, 2	50	yes	466.3	365.5	1.276	373	293 (304)

Residual counts and relation counts are identical in all three arms; every logarithm is solved and correct. S_{rho} means (4 runs): 1.10, 1.15, 1.48, 1.25.

Verdict

- Estimate (210–260k linear, 1.22–1.29x) **beaten**: measured 199–230k linear and 1.27–1.31x C_{cov} (measured). The cheaper L1a variant reaches 241–269k and 1.24–1.26x.
- End to end S improves 1.28x; distance to rho falls from 194–851x to 151–664x. Relation collection stays over 99.9 % of S.
- Sparse beats L1a by 15 % of the linear stage: Wiedemann on the sparse M_{x_6} is cheaper than its Hessenberg form.
- Limits: counted cost only (wall time is observation on a shared host); unpriced native work is not emitted, so S/S_{rho} is a lower-bound comparison; 1 seed per prime for ccov, 2 per prime for dlp. Retries are 4 Wiedemann draws; a non-separating x_6 (1–3 % of tests) wastes them, so 2 draws would cut the linear stage a little more (proposed, not measured).
- Two existing tests, `stopped_f4_agrees_with_the_full_solver_and_the_oracle` and `replayed_f4_agrees_with_the_full_solver_and_the_oracle`, asserted that F4 stops at exactly 64 standard monomials. The extra rng draw of the linear stage shifts their random residuals and brings in one of ideal degree 62 (a solution at infinity), which correctly stops at 62: `stop_staircase` stops at **at most** the bound. Both now assert ≤ 64; the whole `jv_cover` test module passes in release (16 passed, 1 ignored).